Accessibility Report

Ensuring Inclusive and Barrier-Free User Experiences

Introduction

The Web Content Accessibility Guidelines (WCAG) are a series of recommendations developed by the World Wide Web Consortium (W3C) aimed at increasing web accessibility. The guidelines are critical as they provide a framework to help organizations create web content that is more accessible to people with disabilities. As of the latest versions, WCAG 2.2 and 3.1, the recommendations encompass a wide variety of requirements covering everything from text alternatives for non-text content to ensuring keyboard accessibility and proper navigation through web pages. These guidelines are essential in promoting inclusivity, allowing users with different abilities to effectively interact with digital content, and are recognized globally as standard practices for web and application design.

Compliance with WCAG is not only a practice in corporate responsibility but a legal requirement in many regions. For instance, the European Accessibility Act mandates that, starting June 28, 2025, all businesses within the EU must fully comply with accessibility requirements or face significant penalties. Therefore, adhering to these guidelines is critical for legal compliance and for avoiding potential fines and revising projects at a later stage.

This document is designed to help you understand the various accessibility issues present in our current project and provide insights on how to address them effectively. By highlighting specific areas of non-compliance and suggesting detailed solutions, this document aims to facilitate the process of adapting our products to meet global accessibility standards, thus ensuring that we advance towards the inclusive digital experience expected in contemporary digital environments.

Accessibility Issues Found

In our thorough review of the project's codebase, several accessibility issues have been discovered across various components and screens. These issues, if left unaddressed, could pose significant barriers to users with disabilities and are non-compliant with the current Web Content Accessibility Guidelines (WCAG). As accessibility is fundamental in providing inclusive digital experiences, identifying and resolving these issues is crucial.

This section highlights the specific problems found, categorized by file and component, and provides guidance on how to address them. By bringing these issues to the forefront, we aim to ensure that our project not only meets legal requirements but also adheres to best practices in universal design. By aligning with WCAG standards, we can enhance usability, ensure legal compliance, and ultimately create a more welcoming environment for all users.

From issues like the lack of semantic HTML, insufficient keyboard navigation, to inadequate handling of screen reader notifications, each identified problem comes with suggested solutions to enhance accessibility. This proactive approach ensures that necessary measures are taken to correct these issues before they become a larger barrier for inclusivity. Through dedicated efforts towards accessibility, we can foster a digital environment that respects and values all users' needs.

Server/src/screens/index.js

The `index.js` file located in the `server/src/screens` directory serves as an integral part of our project's module export structure. This file currently exports components from several key files: `Login`, `MainMenu`, and `NoAuth`. Each of these components plays a critical role in the project’s navigation and user interaction. However, the abstract nature of this export without further analysis of their individual contents implies that any accessibility issues linked to these components are currently indeterminate from this file alone.

Addressing accessibility compliance requires a detailed examination of the `Login.jsx`, `MainMenu.jsx`, and `NoAuth.jsx` files. By doing so, we can assess how these components align with core guidelines such as providing appropriate text alternatives for non-text elements, ensuring an intuitive and logical focus order across interactive elements, and applying semantic HTML practices.

The adherence to these guidelines supports not only user inclusivity but also aligns with regulatory compliance mandates such as WCAG. Proper auditing of these components will thus require a review tailored to uncover any hidden access barriers and work towards rectifying them. This proactive approach is crucial to maintaining a standard of accessibility that befits modern digital environments.

server/src/screens/NoAuth.jsx

The `NoAuth.jsx` file is crucial for managing unauthenticated user actions within the application. However, it currently utilizes non-semantic HTML elements like `'box'` and `'text'`, which lack the necessary ARIA roles and properties to be accessible by assistive technologies. This can lead to significant accessibility challenges for users who rely on these technologies to navigate and interact with web applications.

Addressing this issue involves replacing these non-semantic elements with well-structured semantic HTML elements. For example, using `<div>` and `<span>` with roles such as `role='alert'` for error messages can meaningfully improve the interface's accessibility. Ensuring that these elements have appropriate ARIA labeling is crucial for conveying the right information about their purpose and content to screen readers and other assistive technology tools.

Adhering to WCAG's guideline 1.3.1 on Info and Relationships will enhance the user experience by maintaining a semantic, understandable structure throughout the interface. This proactive approach is not only compliant with accessibility standards but also ensures our product is inclusive and usable by everyone, regardless of their capabilities.

server/src/screens/NoAuth.jsx

Ensuring accessibility within our application involves critical scrutiny of how messages are presented to users. In the `NoAuth.jsx` file, one significant issue impacting accessibility is the improper delivery of error messages to users relying on screen readers.

The current implementation fails to alert screen readers effectively about error states, making it challenging for visually impaired users to recognize when an error has occurred. This oversight is contrary to WCAG guideline 4.1.2, which emphasizes correct use of name, role, and value to make user interfaces accessible.

To resolve this, we recommend encapsulating the error message within an ARIA live region. Utilizing a `<div role='alert'>` for error messages will ensure that assistive technologies promptly notify users about error states. This adjustment not only complies with accessibility standards but significantly enhances the user experience for those using assistive technologies.

By adopting these practices, we advance our commitment to creating a more inclusive environment, ensuring all users, regardless of their abilities, can navigate, understand, and benefit from our digital products effectively.

server/src/screens/NoAuth.jsx

One critical accessibility barrier within the `NoAuth.jsx` file is the failure to adequately update dynamic content for screen readers. Specifically, the error messages generated dynamically do not refresh in a manner that alerts users relying on assistive technology. This oversight aligns with the WCAG 4.1.2 guideline, which emphasizes the importance of correctly defining and updating the **name, role, and value** of accessible content to ensure that all users receive real-time, relevant notifications about changes.

To address this issue effectively, it is essential to employ **ARIA live regions**. By wrapping the dynamic components or error messages within an ARIA live region, the application can notify screen readers immediately when changes occur. This approach not only enhances real-time feedback for updates but also ensures compliance with accessibility standards, creating a more inclusive experience for users dependent on screen readers.

Correct implementation involves recognizing and labeling these dynamic sections appropriately to convey the changes to assistive technologies. Recommend using elements like `<div role='alert'>` for critical messages that require immediate notifications or `<span aria-live='polite'>` for updates that can be announced with less urgency, depending on the nature of the information.

By incorporating these adjustments, the application will support users more effectively, fulfilling both ethical responsibilities and legal requirements related to digital accessibility.

Server/src/screens/MainMenu.jsx

This section addresses the critical accessibility issues found within the `MainMenu.jsx` file of our application. The primary concern identified is the usage of non-semantic elements such as `<box>` and `<text>` in the render method. These elements lack semantic meaning, leading to significant challenges for screen readers and other assistive technologies, which rely on proper semantics to convey information to users with disabilities.

The non-semantic nature of these elements can disrupt the logical flow and information hierarchy that assistive technologies depend on, causing navigation and understanding issues for users who rely on these tools. Adhering to WCAG Guideline 1.3.1, "Info and Relationships," it is crucial to employ semantic HTML elements that communicate clearly the roles and relationships within the application's interface.

To enhance accessibility, it is recommended to replace `<box>` and `<text>` with more meaningful, semantic HTML tags such as `<div>`, `<header>`, `<main>`, `<section>`, `<article>`, and appropriate heading elements based on the context. This ensures that the document structure is both meaningful and navigable for users relying on screen readers.

Implementing these changes will significantly improve the accessibility of the MainMenu component, ensuring that all users, regardless of their abilities, can interact with it effectively and efficiently. This proactive approach not only aligns with best practices in universal design but also meets legal compliance requirements, contributing to a more inclusive digital environment.

server/src/screens/MainMenu.jsx

The `MainMenu.jsx` component is a critical part of our application's navigation system, offering users access to various features and functionalities. However, a significant accessibility issue identified in this component is the lack of keyboard navigation for interactive elements. According to WCAG Guideline 2.1.1, all interactive elements must be accessible via keyboard to ensure that users who rely on keyboard navigation, including those with physical disabilities, can easily interact with the application.

Currently, some interactive elements within the main menu may not be reachable or operable using just the keyboard, presenting a barrier to accessibility. This oversight can hinder the usability of the application for individuals who depend on keyboard navigation due to the absence of touch or pointer devices.

To address this issue, it’s essential to implement keyboard accessibility throughout the MainMenu component. This involves ensuring that all interactive elements, such as menu items and links, are focusable using `tabindex` attributes where necessary. Furthermore, it is advisable to refactor any non-semantic HTML elements such as `<box>` to more appropriate semantic elements like `<button>` or `<li>` (list item) within a `<ul>` (unordered list) for menu lists. These elements should be enhanced with event handlers to support keyboard events alongside mouse clicks, ensuring they respond correctly when activated with keys such as `Enter` or `Spacebar`.

Implementing these changes guarantees a smooth and intuitive navigation experience for users relying on keyboard access, thereby fostering inclusivity within our application. In compliance with the WCAG, these adjustments not only improve accessibility but also enhance the overall usability of the interface, ensuring that all users can efficiently access the features and content offered by the MainMenu component.

Server/src/screens/MainMenu.jsx

Managing focus effectively is a key component of ensuring accessibility within our application's navigation system, particularly when users transition between different screens using the `this.navigate` method. A potential accessibility issue identified in the `MainMenu.jsx` component is the lack of focus management, which might affect users relying on screen readers or keyboard navigation.

According to WCAG Guideline 2.4.3, 'Focus Order,' it is essential to maintain an intuitive focus order that mirrors the visual order of the page content, ensuring users can follow the navigation path logically and effectively. Without explicit focus management, users who rely on keyboard navigation or screen readers may find it difficult to keep track of their position within the application, potentially missing critical information or becoming disoriented.

To address this issue, focus should be explicitly set to the main content of the page or the first interactive element when navigating between screens. In functional components, the `useRef` hook can be employed, while `createRef` should be used in class components to set the focus programmatically. This approach not only complies with accessibility standards but also improves the overall user experience by providing a seamless and intuitive navigation process.

Implementing these changes will significantly enhance the accessibility and usability of the MainMenu component, ensuring that users, regardless of their abilities, have equivalent access to the application's features and content. By prioritizing focus management, we advance our commitment to fostering an inclusive digital environment, in line with both ethical responsibilities and legal compliance requirements.

server/src/screens/Login.jsx

The `Login.jsx` file in the `server/src/screens` directory plays a crucial role in our application's authentication process. A significant issue identified pertains to the dynamic text counter that displays the "Count:" value. This element does not update in a way that screen readers can detect, resulting in accessibility barriers for users who rely on assistive technology to interact with our application.

This issue contravenes WCAG Guideline 4.1.3 on Status Messages, which emphasizes that changes in content dynamically should be presented in a manner that allows screen readers to announce updates effectively. Failing to update the "Count:" text dynamically may cause users to miss critical information, thereby impairing their ability to interact effectively with the application.

To resolve this issue, it is recommended to wrap the "Count:" text in an ARIA live region. By enclosing this dynamic content in a `<span aria-live="polite">`, any changes will be automatically announced by screen readers when they occur. This ensures users receive immediate feedback regarding their actions without requiring additional navigation or interaction.

Implementing this fix will not only adhere to accessibility standards but also significantly enhance user experience for all users, promoting inclusivity within our application. It is crucial to continually assess and update these dynamic elements as needed, ensuring full compliance with accessibility guidelines and improving the overall usability of the interface.

server/src/screens/Login.jsx

A critical accessibility concern within the `Login.jsx` component is the lack of keyboard accessibility for the form's submission button. According to WCAG Guideline 2.1.1, ensuring that all interactive elements, such as buttons, are operable through keyboard interfaces is essential. This functionality is crucial for users who rely on keyboard navigation due to disabilities that impede the use of a mouse or other pointing devices.

Presently, the login form's submit button might not be reachable or operable using only keyboard commands, creating accessibility barriers. This oversight can significantly affect the usability and inclusivity of the application, particularly for users with physical disabilities or those using assistive technologies.

To improve keyboard accessibility, it is essential to ensure that the submission button can be activated not only by mouse clicks but also via keyboard input. This can be achieved by adding event handlers such as `onKeyPress` or configuring the button with `type="submit"` to respond correctly to the `Enter` key. Furthermore, the button should be focusable, which can be enhanced using the `tabindex` attribute if necessary, allowing seamless navigation within the form through keyboard inputs.

Implementing these changes aligns with accessibility standards and enhancements, providing an inclusive experience that empowers all users, regardless of their physical abilities, to interact with the application effectively. By adhering to these guidelines, we ensure that the Login component is accessible, thereby supporting a broader spectrum of user needs and expectations.

server/src/screens/Login.jsx

Within the `Login.jsx` component, a critical accessibility issue involves the use of images such as the logo without proper alternative text. This oversight can create barriers for users who rely on screen readers, as these technologies need meaningful descriptions to convey the content of images to users with visual impairments.

According to WCAG Guideline 1.1.1 on Non-text Content, each image must have descriptive alternative text to be accessible. Failing to do so means that users who cannot see the images will miss out on important visual information, potentially affecting their interpretation and usability of the application.

To remedy this, each image should be endowed with the `alt` attribute that succinctly and accurately describes its content. For instance, the logo in question can be coded as `<ansiimage src="./assets/logo.gif" alt="Company Logo" animated="true" />`, ensuring that the description provided via the `alt` attribute is conveyed to users by screen readers.

Implementing these changes not only aligns with accessibility standards, ensuring compliance with WCAG, but also enhances the user experience by making the application more inclusive. Providing alternative text is a straightforward yet impactful practice that demonstrates a commitment to accessibility and inclusivity for all users.

server/src/screens/Login.jsx

Within the `Login.jsx` component, effectively handling user input errors is crucial for ensuring that users with disabilities are informed in a timely and comprehensible manner. Currently, there is a significant gap in how incorrect user inputs are communicated, particularly to those using assistive technologies such as screen readers.

As stipulated in WCAG Guideline 3.3.3, applications should provide error suggestions that help users remedy the mistakes they've made. However, the current implementation may not adequately support this, as error messages related to incorrect inputs are not marked up specifically for screen readers. This lack of accessibility can lead to confusion and a frustrating experience for users who rely on these technologies.

To amend this, error messages associated with login failures should be enriched using ARIA attributes to ensure they’re correctly announced by assistive technologies. For example, wrapping the error messages in an `aria-live` region, specifically using a `<div role="alert">`, will enable screen readers to automatically announce the errors as soon as they occur. This practice not only promotes a more accessible interface but also aligns with accessibility standards by providing users with real-time feedback and guidance on correcting their inputs.

By integrating these accessibility features, we can ensure our platform is inclusive and caters to the needs of all users, ultimately enhancing their experience and interaction with our application.

server/src/components/test.jsx

Within the `test.jsx` component, a key accessibility issue concerns the use of `<p>` tags to contain interactive content. This practice poses challenges for screen reader users and other individuals relying on assistive technologies, as it may not appropriately convey the interactive nature of the content.

Standard accessibility guidelines, such as WCAG 1.3.1, emphasize the importance of using semantic HTML elements that clearly represent the intended function of the content. In this case, elements like `<button>` or `<span>` with appropriate `role` and `tabindex` attributes should be used to denote interactive content. This helps ensure that assistive technologies can accurately interpret and relay the user's options.

To address this issue, consider refactoring the interactive portions of the `test.jsx` component by replacing `<p>` tags with elements that inherently or contextually provide information about their purpose. Applying ARIA roles where necessary can also enhance the accessibility of these elements, ensuring they communicate their interactivity to users who depend on screen readers.

Implementing these changes will align the `test.jsx` component with best practices in accessibility, fostering an inclusive experience for all users. This improvement not only supports ethical digital design but also complies with legal accessibility requirements.

server/src/components/test.jsx

Within the `test.jsx` component, the interactive content 'hello: {props.name}' does not provide adequate context for users relying on screen readers. As per the Web Content Accessibility Guidelines (WCAG) 1.3.1, it is essential that content conveys relationships and context accurately to assistive technologies to ensure users with disabilities receive the same information as those who do not use screen readers.

The lack of descriptive annotations for the interactive text content can create confusion and hinder the user's understanding of its purpose within the user interface. This oversight might lead to an incomplete interaction experience for individuals relying on assistive technologies.

To enhance accessibility, it is recommended to include `aria-label` or `aria-labelledby` attributes on the containing element. These attributes provide a textual description that screen readers can use to convey the intended meaning of the content effectively. For example, using `aria-label='Greeting message for the user'` could give screen reader users a better understanding of the content's purpose.

By implementing these changes, we make the application more inclusive, ensuring that all users, regardless of ability, can perceive and interact with the interface in a meaningful way. This approach not only aligns with legal accessibility standards but also promotes an equitable user experience.